

# CTF Writeup: Return the Blow — Broken Trust

JWT Secret Cracking + PNG Steganography

**Author:** Senpai

**Category:** Web / JWT / Steganography

**Points:** 497

**Flag:** SK-CERT{why\_15\_h3\_h3r3}

---

## Context

From the previous challenge we knew the attacker's exfil server was `http://g00gl3.onLine:7050`. This challenge asks us to turn the tables and dig into that server.

## Step 1 — Recon the Target

Navigating to `http://g00gl3.onLine:7050/app` revealed a login page. Using the credentials stolen by the backdoor from the previous challenge — `demo:password123` — we logged in as a USER role account. The app issued a JWT via Set-Cookie:

```
token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJkZW1vIiwicm9sZSI6InVzZXIifQ.HoKfNvb5HbclC2ikvtWxp2Y012hDrH_hs2n2MWNjpLc
```

```
Decoded payload: {"sub":"demo","role":"user"}
```

A "View source" link at the bottom pointed to `/source`.

## Step 2 — Source Code Disclosure

`/source` listed all server-side files openly. Reading `server.js` revealed the flag lives at `/api/admin/data` gated behind `requireAdmin` which checks `req.user.role === 'admin'`. We need an admin JWT.

## Step 3 — The Leaked Token

Reading `auth.js` was the key finding. The developer had hardcoded a blacklisted admin token with a comment explaining why:

```
function verify(tok) {
  // One of the admins was stupid and leaked their token, so refuse it.
  if(tok == "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJkZW1vIiwicm9sZSI6InVzZXIifQ.HoKfNvb5HbclC2ikvtWxp2Y012hDrH_hs2n2MWNjpLc")
    return false;
  ...
  if(sb64 == "LX+atl-MwNSvuTpqYnhDiNe3UBX1BwDBH+iQ/r/0258") return false;
```

Decoding the payload: `{"sub":"admin","role":"admin"}` — a valid admin token, signed with the real `JWT_SECRET`. If we crack the secret we can forge a new admin token with a different signature that bypasses both blacklist checks.

## Step 4 — Cracking the JWT Secret

With the leaked token in hand, we used hashcat mode 16500 (JWT HS256 cracking) against `rockyou.txt`:

```
echo 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbi...' > jwt.txt
hashcat -a 0 -m 16500 jwt.txt /usr/share/wordlists/rockyou.txt
```

This recovered the secret, allowing us to forge a fresh admin token with identical payload but a different signature — bypassing both blacklist checks.

## Step 5 — The Hidden File

While reviewing `server.js` more carefully, `VIEWABLE_SOURCES` contained an entry deliberately excluded from the `/source` index page listing:

```
'spunchbob.png' // listed but filtered out of the UI
```

Fetching it directly:

```
curl -s -b "token=$TOKEN" http://g00gl3.online:7050/source/spunchbob.png -o spunchbob.png
```

Running `strings` on it immediately revealed:

```
tEXtFlag
SK-CERT{why_15_h3_h3r3}
```

The flag was embedded as a PNG `tEXt` metadata chunk — a standard steganography-lite technique where arbitrary key-value data is stored in PNG ancillary chunks, invisible to the naked eye but trivially found with `strings` or a PNG chunk parser.

## Flag

```
SK-CERT{why_15_h3_h3r3}
```

## Key Takeaways

- Never expose source code in production — once an attacker has any valid session they can read your entire codebase.
- Blacklisting tokens by value is not revocation — the secret is still compromised. Rotate the secret; don't patch around a leak.
- Hidden  $\neq$  secure — filtering a file from a UI listing while keeping it accessible via direct URL is security through obscurity.
- PNG `tEXt` chunks are a classic CTF hiding spot. Always run `strings` and `exiftool` on any image in a challenge.